

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования

«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ И СЕТЕЙ

ОЦЕНКА

ПРЕПОДАВАТЕЛЬ

Старший преподаватель
должность, уч. степень, звание

подпись, дата

Д.А.Булгаков
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Лабораторная работа: работае с дисплеем

по дисциплине: Управление виртуальным двойником

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4241в
номер группы

подпись, дата

А.А.Хусаинов
инициалы, фамилия

Студенческий билет №

2022/2783

Санкт-Петербург 2026

Цель работы

Цель работы — реализовать управление OLED-дисплеем SSD1306 128×64 с Arduino из веб-интерфейса, организовать обмен данными через COM-порт с использованием Web Serial API и отобразить содержимое OLED не только на физическом дисплее, но и в интерфейсе цифрового двойника на Three.js.

В ходе работы решались следующие задачи:

- настроить прошивку Arduino для приёма команд по Serial;
- реализовать команды для вывода текста, чтения текущего состояния, отображения углов и передачи bitmap;
- подключить веб-интерфейс к Arduino через Web Serial API;
- вывести состояние OLED в HTML canvas;
- сформировать texture для 3D-модели, чтобы экран в Three.js показывал те же данные, что и OLED.

Используемое оборудование и программные средства

В проекте используется плата Arduino Nano на реальной установке. Для отладки также применялась тестовая Arduino Uno без подключенных сервоприводов и OLED-дисплея.

Основные компоненты:

- Arduino Nano / Arduino Uno;
- OLED-дисплей SSD1306 128×64, I2C-адрес 0x3C;
- веб-приложение на Vite + TypeScript;
- Web Serial API в браузере Chrome/Edge;
- Three.js для 3D-визуализации;
- Arduino-библиотеки Servo, Wire, Adafruit_GFX, Adafruit_SSD1306.

Общая схема работы

Система состоит из трёх основных частей:

1. Arduino-прошивка принимает команды по Serial, управляет OLED и возвращает ответы.
2. Веб-интерфейс отправляет команды через Web Serial API и отображает TX/RX лог.
3. Three.js-модуль рисует содержимое OLED в canvas и использует его как texture для mesh OLED_SCREEN на 3D-модели.

Логическая схема обмена:

Пользователь

- > Веб-интерфейс
- > Web Serial API
- > USB COM-порт
- > Arduino
- > OLED SSD1306

Arduino

- > Serial-ответ
- > Web Serial API
- > OLED preview canvas
- > Three.js CanvasTexture

COM-порт и Web Serial API

COM-порт — это последовательный порт, через который компьютер обменивается байтами с внешним устройством. В Linux подключённая Arduino обычно отображается как устройство вида `/dev/ttyUSB0` или `/dev/ttyACM0`. В Windows это обычно COM3, COM4 и т.п.

Для Arduino последовательный порт является стандартным способом отладки и управления. В прошивке он открывается строкой:

```
Serial.begin(9600);
```

Скорость 9600 baud означает, что обе стороны должны работать на одной скорости. Если браузер откроет порт на другой скорости, текстовые команды и ответы будут читаться неправильно.

Обычная веб-страница не имеет прямого доступа к COM-портам из соображений безопасности. Поэтому используется Web Serial API. Этот API доступен в Chromium-браузерах, например Chrome и Edge. Пользователь обязательно должен вручную выбрать устройство в диалоге браузера.

Фрагмент подключения в `src/serial.ts`:

```
export async function connect(baudRate: number): Promise<void> {  
  if (!isSupported()) {  
    throw new Error('Web Serial API не поддерживается в этом браузере')  
  }  
  
  port = await navigator.serial.requestPort()  
  await port.open({ baudRate })  
  
  reader = port.readable!.getReader()  
  writer = port.writable!.getWriter()  
  
  readLoopActive = true  
  readLoop()  
}
```

Здесь `navigator.serial.requestPort()` открывает системный диалог выбора порта. После выбора вызывается `port.open({ baudRate })`, затем приложение получает `reader` для чтения входящих байтов и `writer` для отправки команд.

Отправка данных выполняется через `TextEncoder`:

```
export async function send(data: string): Promise<void> {  
  if (!writer) {  
    throw new Error("Не подключено")  
  }  
}
```

```
await writer.write(textEncoder.encode(data))  
}
```

Проверка на реальном стенде

Проверим получение и отправку текста из интерфейса для отображения на oled экране.

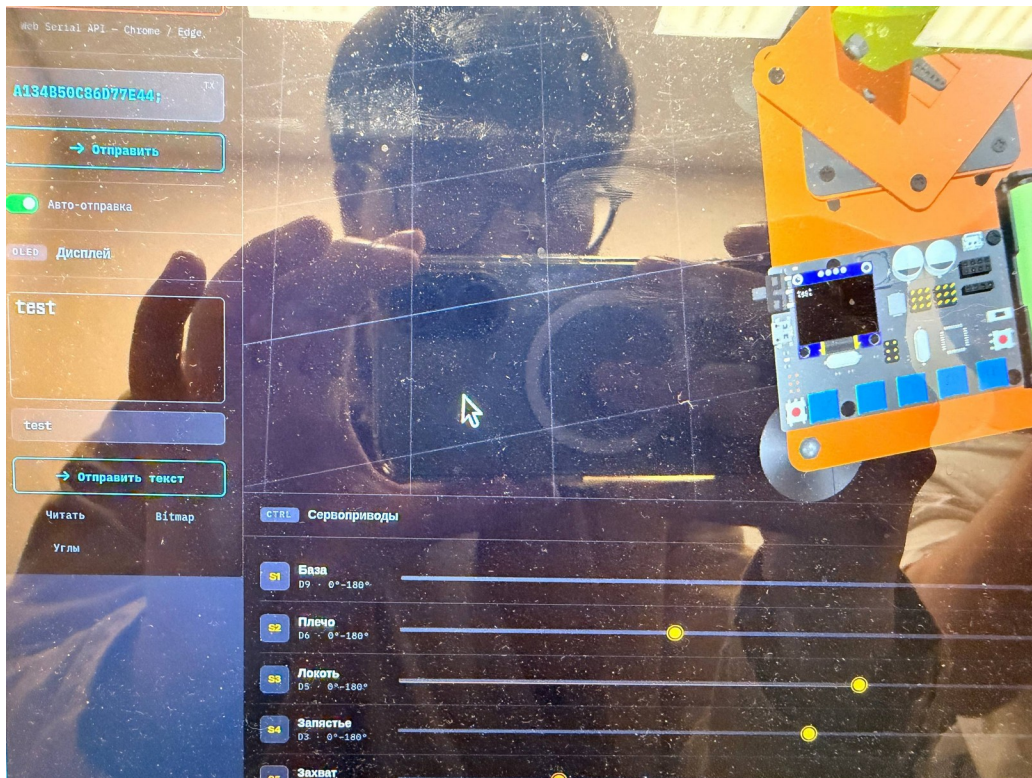


Рис. 1 — интерфейс с тестовой отправкой

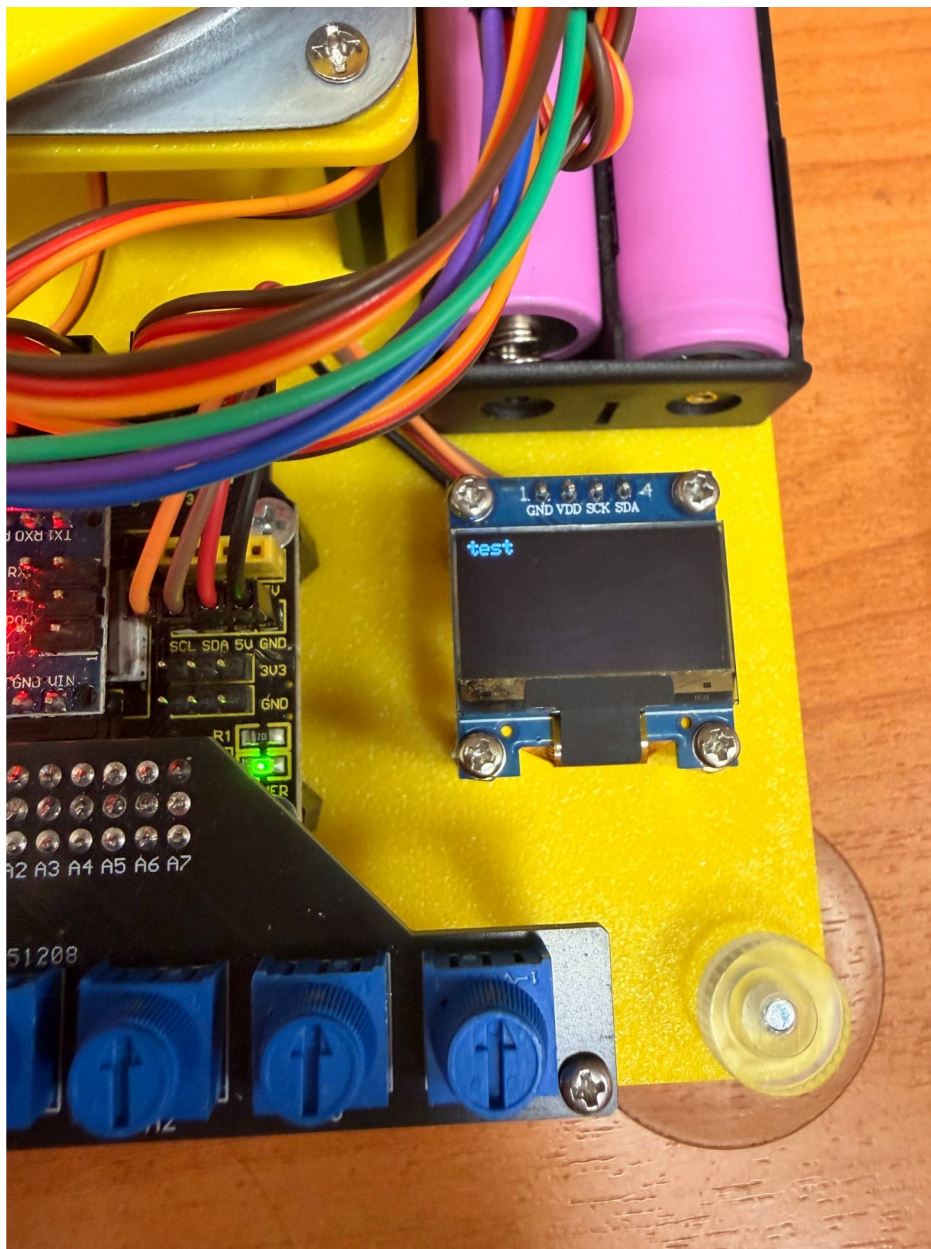


Рис. 2 — oled экран с тестовой отправкой

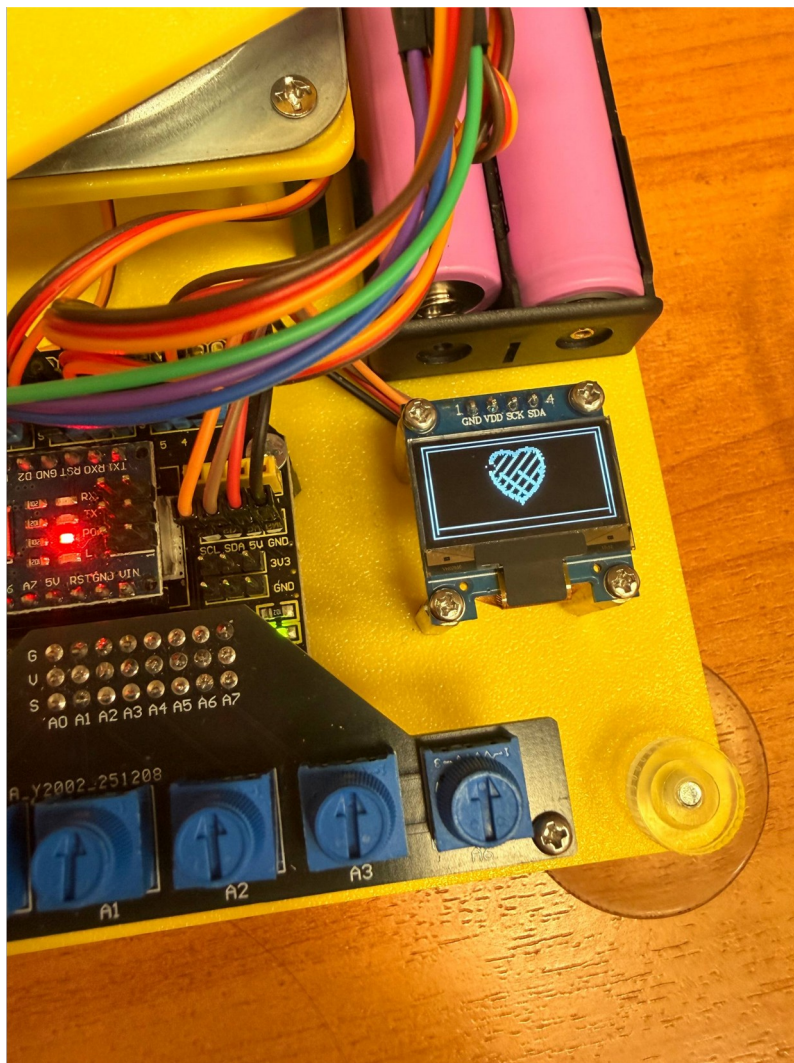


Рис. 3 — oled экран с bitmap изображением

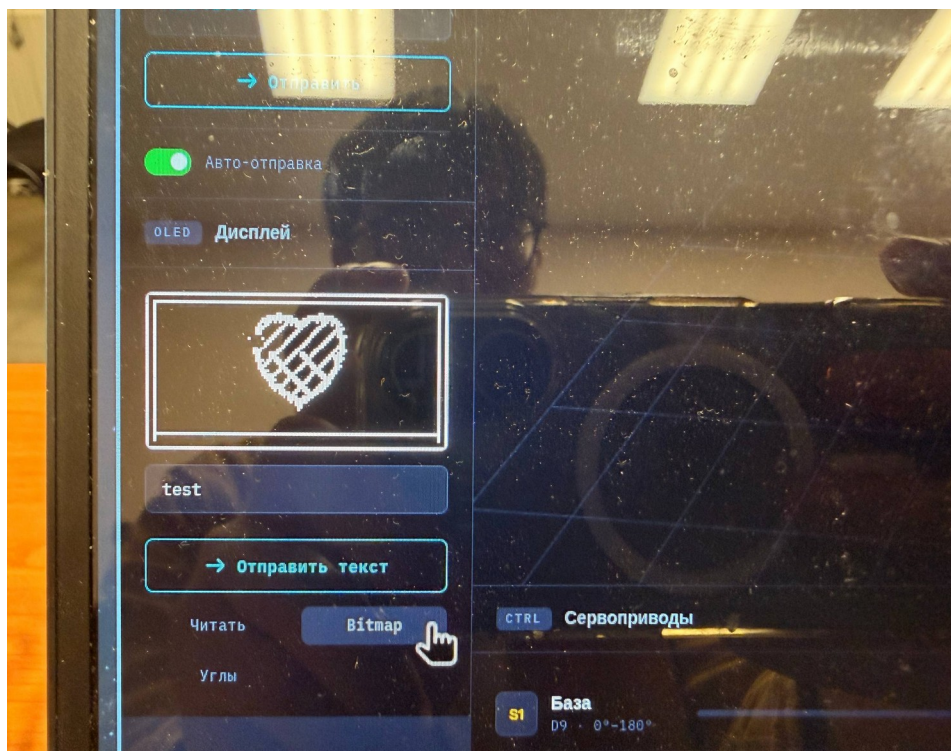


Рис. 4 — интерфейс с отображением битмар



Рис. 5 — интерфейс с углами



Рис. 6 — oled экран с углами

Протокол OLED-команд

Команды в проекте имеют текстовый формат и завершаются символом ;. Arduino читает входящий поток до ;, после чего разбирает команду.

Поддерживаемые команды:

Команда	Назначение	Пример ответа
OLED_TEXT:<text>;	Показать текст на OLED	%Interface OLED test%;
OLED_READ?;	Прочитать текущий OLED-текст	%Khusainov AA 4241v%;
OLED_ANGLES?;	Включить режим отображения углов сервоприводов	%A90 B90 C90 D90 E90%;
OLED_BITMAP?;	Показать bitmap и передать его в веб-интерфейс	#OLED_BITMAP:1024 + 1024 байта

Текстовый ответ Arduino помещается между символами %. Это упрощает извлечение OLED-текста на стороне веб-интерфейса:

```
function extractOledText(input: string): string | null {  
  const firstIndex = input.indexOf('%')  
  if (firstIndex === -1) return null  
  
  const secondIndex = input.indexOf('%', firstIndex + 1)  
  if (secondIndex === -1) return null  
  
  return input.slice(firstIndex + 1, secondIndex)  
}
```

Реализация на Arduino

В прошивке используется фиксированный буфер команд. Это безопаснее для Arduino Nano/Uno, чем частое использование String, потому что у AVR-плат всего 2 KB SRAM, и динамические строки могут фрагментировать память.

Основные настройки прошивки:

```
#define SCREEN_WIDTH 128  
#define SCREEN_HEIGHT 64  
#define OLED_RESET -1  
#define OLED_ADDRESS 0x3C  
#define OLED_REQUIRED false  
#define SERIAL_BAUD 9600
```

```
#define COMMAND_BUFFER_SIZE 97
#define OLED_TEXT_BUFFER_SIZE 81
```

Если OLED отсутствует, программа продолжает работать:

```
displayAvailable = display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS);
```

```
if (!displayAvailable) {
    Serial.println("WARN SSD1306 not found, mock mode");
    if (OLED_REQUIRED) {
        Serial.println("ERR SSD1306 init failed");
        while (true) {
            delay(1000);
        }
    }
}
```

Вывод текста на OLED:

```
void showText(const char *text) {
    if (!displayAvailable) return;

    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    display.setTextSize(1);
    display.setCursor(0, 0);
    display.print(text);
    display.display();
}
```

Отправка текущего текста обратно в веб-интерфейс:

```
void sendCurrentText() {
    Serial.print('%');
    Serial.print(currentText);
    Serial.println("%;");
}
```

Обработка OLED-команд:

```
if (strcmp(command, "OLED_TEXT:", 10) == 0) {
    oledMode = OLED_MODE_TEXT;
    strncpy(currentText, command + 10, sizeof(currentText) - 1);
    currentText[sizeof(currentText) - 1] = '\0';
    trimCommand(currentText);
    showText(currentText);
    sendCurrentText();
    return;
}
```

```
if (strcmp(command, "OLED_READ?") == 0) {
    sendCurrentText();
}
```

```
    return;  
}
```

Передача bitmap устроена иначе: сначала Arduino отправляет текстовый заголовок, затем 1024 бинарных байта. 1024 байта соответствуют монохромному изображению 128×64, где 1 бит описывает 1 пиксель.

```
void sendImage() {  
    const int bufferSize = 32;  
    byte buffer[bufferSize];  
  
    Serial.print("#OLED_BITMAP:");  
    Serial.println(myBitmapSize);  
  
    for (unsigned int i = 0; i < myBitmapSize; i += bufferSize) {  
        int chunkSize = min(bufferSize, (int)(myBitmapSize - i));  
        memcpy_P(buffer, &myBitmap[i], chunkSize);  
        Serial.write(buffer, chunkSize);  
    }  
}
```

Реализация в веб-интерфейсе

В интерфейсе есть OLED-панель:

- canvas preview 128×64;
- поле ввода текста;
- кнопки Отправить текст, Читать, Bitmap, Углы;
- Serial Monitor с TX/RX логом.

Фрагмент HTML:

```
<canvas  
  class="oled-preview"  
  id="oledPreview"  
  width="128"  
  height="64"  
  aria-label="OLED preview"  
></canvas>  
  
<input  
  type="text"  
  class="monitor-input__field oled-input"  
  id="oledTextInput"  
  maxlength="80"  
  aria-label="Текст OLED"  
>
```

Обработчик отправки текста:


```

async function sendTextToOled() {
  const text = input?.value.trim() || ""
  if (!text) return

  setOledAnglesMode(false)
  showOledText(text)
  await sendCommand(`OLED_TEXT:${text};`)
}

```

Обработчик входящих текстовых строк:

```

serial.onReceive((data) => {
  lastReceivedLine = data
  addLogEntry('rx', data)

  const oledText = extractOledText(data)
  if (oledText !== null) {
    showOledText(oledText)
  }
})

```

Для bitmap используется отдельный callback:

```

serial.onBitmapReceive((data) => {
  showOledBitmap(data)
  addLogEntry('rx', `OLED bitmap received: ${data.length} bytes`)
})

```

Формирование OLED-текстуры в Three.js

В проекте OLED отображается не только на физическом дисплее и в HTML preview, но и на 3D-модели роборуки. Для этого используется canvas 128×64, совпадающий с разрешением SSD1306.

В src/viewer3d.ts задаются размеры OLED и имя mesh в GLB-модели:

```

const OLED_WIDTH = 128
const OLED_HEIGHT = 64
const OLED_SCREEN_NAME = "OLED_SCREEN"

```

Сначала создаётся offscreen canvas:

```

function ensureOledCanvas() {
  if (oledCanvas && oledCtx) return

  oledCanvas = document.createElement("canvas")
  oledCanvas.width = OLED_WIDTH
  oledCanvas.height = OLED_HEIGHT
  oledCtx = oledCanvas.getContext("2d")

  if (oledCtx) {
    oledCtx.imageSmoothingEnabled = false
  }
}

```

```

}
}

```

Если приходит текст, он переносится по строкам и рисуется белым цветом на чёрном фоне:

```

function drawTextToContext(text: string, ctx: CanvasRenderingContext2D) {
  ctx.fillStyle = "#fff"
  ctx.font = "10px monospace"
  ctx.textBaseline = "top"

  const lines = wrapOledText(text)
  lines.slice(0, 5).forEach((line, index) => {
    ctx.fillText(line, 4, 4 + index * 12)
  })
}

```

Если приходит bitmap, каждый бит преобразуется в RGBA-пиксель:

```

function drawBitmapToContext(data: Uint8Array, ctx: CanvasRenderingContext2D) {
  const imageData = ctx.createImageData(OLED_WIDTH, OLED_HEIGHT)

  for (let y = 0; y < OLED_HEIGHT; y++) {
    for (let x = 0; x < OLED_WIDTH; x++) {
      const byteIndex = y * (OLED_WIDTH / 8) + Math.floor(x / 8)
      const bit = 7 - (x % 8)
      const on = (data[byteIndex] & (1 << bit)) !== 0
      const pixelIndex = (y * OLED_WIDTH + x) * 4
      const value = on ? 255 : 0

      imageData.data[pixelIndex] = value
      imageData.data[pixelIndex + 1] = value
      imageData.data[pixelIndex + 2] = value
      imageData.data[pixelIndex + 3] = 255
    }
  }

  ctx.putImageData(imageData, 0, 0)
}

```

Далее этот canvas становится текстурой Three.js:

```

oledTexture = new THREE.CanvasTexture(oledCanvas)
oledTexture.colorSpace = THREE.SRGBColorSpace
oledTexture.magFilter = THREE.NearestFilter
oledTexture.minFilter = THREE.NearestFilter
oledTexture.flipY = false

oledScreen.material = new THREE.MeshBasicMaterial({
  map: oledTexture,

```

```
toneMapped: false,  
})
```

После каждого изменения текста или bitmap выполняется:

```
if (oledTexture) {  
  oledTexture.needsUpdate = true  
}
```

Это сообщает Three.js, что canvas изменился и texture нужно обновить на GPU.

Важно, что HTML preview и 3D-экран используют один и тот же источник изображения. Функция renderOledPreview просто копирует offscreen canvas в видимый canvas интерфейса:

```
export function renderOledPreview(canvas: HTMLCanvasElement) {  
  ensureOledCanvas()  
  if (!oledCanvas) return  
  
  canvas.width = OLED_WIDTH  
  canvas.height = OLED_HEIGHT  
  const ctx = canvas.getContext("2d")  
  if (!ctx) return  
  
  ctx.imageSmoothingEnabled = false  
  ctx.clearRect(0, 0, canvas.width, canvas.height)  
  ctx.drawImage(oledCanvas, 0, 0)  
}
```

За счёт этого физический OLED, OLED preview в интерфейсе и экран на 3D-модели отображают один и тот же смысловой контент.

Результаты проверки

Проверка выполнялась на Arduino Uno без подключенного OLED-дисплея. Это позволило проверить Serial-протокол и mock-mode.

После подключения через Web Serial API были получены стартовые сообщения:

```
WARN SSD1306 not found, mock mode  
OK OLED mock ready  
%Khusainov AA 4241v%;
```

Команда отправки текста из интерфейса:

```
TX OLED_TEXT:Interface OLED test;  
RX %Interface OLED test%;
```

Команда чтения текущего текста:

```
TX OLED_READ?;  
RX %Interface OLED test%;
```

Режим отображения углов:

```
TX OLED_ANGLES?;  
RX %A90 B90 C90 D90 E90%;
```

Передача bitmap:

```
TX OLED_BITMAP?;  
RX OLED bitmap received: 1024 bytes
```

Вывод

В работе реализован полный контур управления OLED-дисплеем через веб-интерфейс. Arduino принимает текстовые команды по COM-порту, управляет OLED SSD1306 или работает в mock-mode без физического дисплея. Web Serial API позволяет браузеру Chrome/Edge напрямую обмениваться данными с Arduino после явного выбора порта пользователем.

Отдельно реализовано отображение OLED-содержимого в Three.js: данные рисуются в canvas 128×64, затем этот canvas используется как THREE.CanvasTexture для mesh OLED_SCREEN. Благодаря этому интерфейс и 3D-модель показывают то же содержимое, которое выводится или должно выводиться на OLED-дисплей.